Lab Activity: Implementation and Analysis of Addressing Modes

1. Objective

To understand and identify various x86-64 addressing modes by analyzing the disassembly of a C program using GDB.

2. Source Code

The following C code was used to demonstrate different memory access patterns:

```
  GNU nano 7.2
#include <stdio.h>
int main() {
int a = 100;

int b = a;

int numbers[5] = {10, 20, 30, 40, 50};
int index = 2;

int value = numbers[index];
int *ptr = &a;
int deref = *ptr;
printf("Results: a=%d, value=%d, deref=%d\n", a, value, deref);
return 0;
}
```

3. Methodology

1. Compilation: Compiled the source with debugging symbols enabled: gcc -g -o test testfile.c
2. Debugging: Loaded the executable into GDB: gdb ./test
3. Disassembly: Set disassembly flavor to Intel and used the disassemble /m command to view C code alongside its assembly instructions.

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test...
(gdb) set disassembly-flavor intel
(gdb) break main
Breakpoint 1 at 0x1175: file testfile.c, line 2.
(gdb) run
Starting program: /home/nitdelhi/Desktop/Aniket/test

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
```

```
nitdelhi@slave1:~/Desktop/Aniket$ gcc -g -o test testfile.c
nitdelhi@slave1:~/Desktop/Aniket$ gdb ./test
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.
```

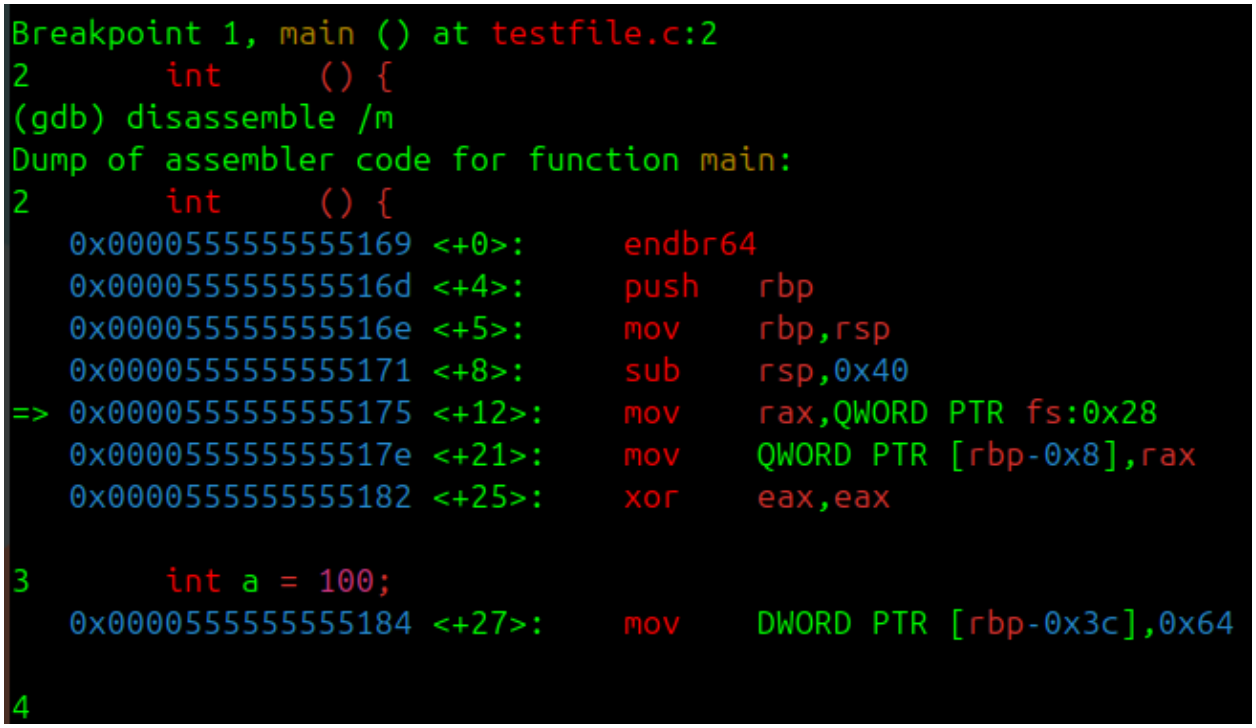
4. Observations & Analysis

From the GDB output, we identified the following addressing modes:

A. Immediate Addressing

Used when a constant value is moved directly into a register or memory location.

- C Code: int a = 100;
- Assembly: mov DWORD PTR [rbp-0x3c], 0x64
- Explanation: The value 0x64 (decimal 100) is moved directly into a stack location.



B. Register Indirect / Base Displacement Addressing

Used to access local variables on the stack using a base register (rbp) and an offset.

- C Code: int b = a;
- Assembly:
    1. mov eax, DWORD PTR [rbp-0x3c] (Loads a into register)
    2. mov DWORD PTR [rbp-0x38], eax (Moves value from register to b)
- Explanation: [rbp-0x3c] uses a base register plus a constant displacement to find the memory address.

C. Scaled Indexed Addressing (SIB)

Used for array indexing.

- C Code: int value = numbers[index];
- Assembly: mov eax, DWORD PTR [rbp+rax*4-0x20]
- Explanation: This implements the formula: $Base + (Index \times Scale) + Displacement$.
    - rbp is the base.

- rax contains the index (2).
- 4 is the scale (size of int).
- -0x20 is the displacement to the start of the array.

```
5       int b = a;
   0x000055555555518b <+34>:    mov    eax,DWORD PTR [rbp-0x3c]
   0x000055555555518e <+37>:    mov    DWORD PTR [rbp-0x38],eax

6
7       int numbers[5] = {10, 20, 30, 40, 50};
   0x0000555555555191 <+40>:    mov    DWORD PTR [rbp-0x20],0xa
   0x0000555555555198 <+47>:    mov    DWORD PTR [rbp-0x1c],0x14
   0x000055555555519f <+54>:    mov    DWORD PTR [rbp-0x18],0x1e
--Type <RET> for more, q to quit, c to continue without paging--c
   0x00005555555551a6 <+61>:    mov    DWORD PTR [rbp-0x14],0x28
   0x00005555555551ad <+68>:    mov    DWORD PTR [rbp-0x10],0x32

8       int index = 2;
   0x00005555555551b4 <+75>:    mov    DWORD PTR [rbp-0x34],0x2

9
10      int value = numbers[index];
   0x00005555555551bb <+82>:    mov    eax,DWORD PTR [rbp-0x34]
   0x00005555555551be <+85>:    cdqe
   0x00005555555551c0 <+87>:    mov    eax,DWORD PTR [rbp+rax*4-0x20]
   0x00005555555551c4 <+91>:    mov    DWORD PTR [rbp-0x30],eax

11      int *ptr = &a;
   0x00005555555551c7 <+94>:    lea    rax,[rbp-0x3c]
   0x00005555555551cb <+98>:    mov    QWORD PTR [rbp-0x28],rax

12      int deref = *ptr;
   0x00005555555551cf <+102>:   mov    rax,QWORD PTR [rbp-0x28]
   0x00005555555551d3 <+106>:   mov    eax,DWORD PTR [rax]
   0x00005555555551d5 <+108>:   mov    DWORD PTR [rbp-0x2c],eax
```

D. Indirect Addressing (Pointer Dereferencing)

- C Code: int deref = *ptr;
- Assembly:
    1. mov rax, QWORD PTR [rbp-0x28] (Loads the address stored in ptr into rax)
    2. mov eax, DWORD PTR [rax] (Accesses the value at the address held in rax)

- Explanation: [rax] tells the CPU to go to the memory address contained inside the rax register.

5. Conclusion

Through this lab, I successfully mapped high-level C constructs (variables, arrays, and pointers) to their low-level assembly addressing modes. The use of GDB's disassemble /m command provided a clear visual mapping between the source code logic and the hardware-level memory execution.